Software Requirements Document

Color Clash - A Two-Player Fighting Game

TOBB ETU – BIL 482

February 2026

Team Members

Mehmet Umur ÖZÜ

Mehmet Yasin TOSUN

Mustafa GÖZÜTOK

Table of Contents

1. System Overview
2. System Context
3. Key Features and Functionality
4. Assumptions and Dependencies
5. Architectural Design
    - 5.1 System Architecture Diagram
    - 5.2 Architectural Patterns and Styles
    - 5.3 Rationale for Architectural Decisions
6. Component Design
    - 6.1 Subsystems and Modules
    - 6.2 Responsibilities of Each Component
    - 6.3 Component Diagrams
7. Data Design
    - 7.1 Data Model
    - 7.2 Data Flow Diagrams
8. Design Patterns
    - 8.1 Applied Design Patterns
    - 8.2 Context and Justification
9. Implementation Notes
10. User Interface Design
11. Performance Considerations
12. Error Handling and Logging
13. Deployment and Installation Design
14. Change Log
15. Future Work / Open Issues
16. Task Assignment Matrix

1. System Overview

Color Clash is a web-based 2D fighting game developed using TypeScript and HTML5 Canvas. Unlike traditional high-violence fighting games, this project emphasizes a vibrant, child-friendly aesthetic and strategic depth. It serves as a technical case study for implementing advanced software design patterns in a real-time game engine environment.

2. System Context

The application is a standalone, client-side web application.

- **Platform:** Desktop/Laptop web browsers (Chrome, Firefox, Safari).
- **Technology Stack:** TypeScript (Logic), HTML5 Canvas (Rendering), Vite (Build System).
- **External Interfaces:** Browser DOM for UI and the Keyboard API for user input.

3. Key Features and Functionality

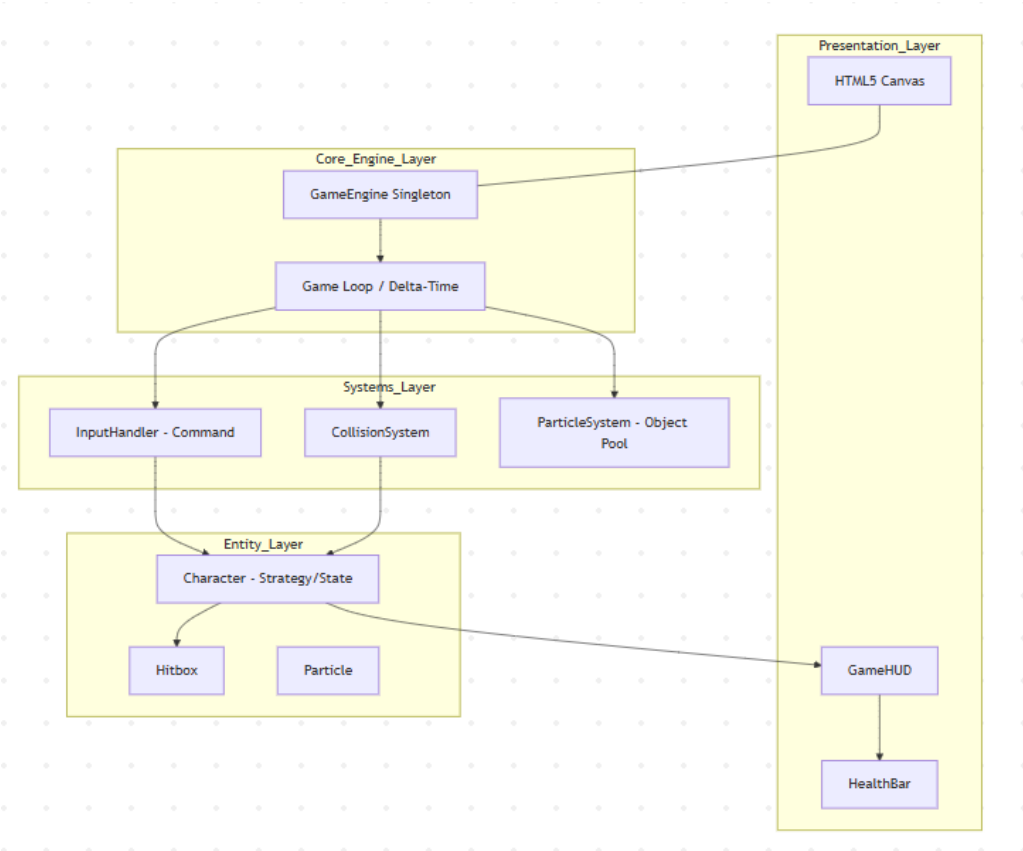
- **UC1 - Start Game Battle:** Initialization of the singleton game engine, character instantiation via a Factory, and input listener setup.
- **UC2 - Execute Combat Actions:** Real-time movement (left, right, jump), collision resolution, and hitbox-based combat.
- **Elemental Strategy:** The ability to toggle between "Fire" (High Damage) and "Water" (High Defense) modes, affecting combat attributes dynamically.

4. Assumptions and Dependencies

- **Assumptions:** Players interact via a single shared keyboard for local multiplayer.
- **Dependencies:** Requires Node.js for development, TypeScript for compilation, and Vite for the development server and bundling.

5.1 System Architecture Diagram

The system follows a Layered Architecture to decouple game logic from the rendering hardware.



5.2 Architectural Patterns and Styles

- **Singleton Pattern:** Applied to the GameEngine to ensure a single authoritative source for the game loop and state management.
- **Layered Style:** Separates core combat physics from the UI/Rendering logic, facilitating independent testing.

5.3 Rationale for Architectural Decisions

The Singleton ensures that multiple engine instances do not compete for the CPU or Canvas context. The Layered Architecture allows for future-proofing; for instance, the rendering layer could be swapped for WebGL without modifying the core physics logic.

6. Component Design

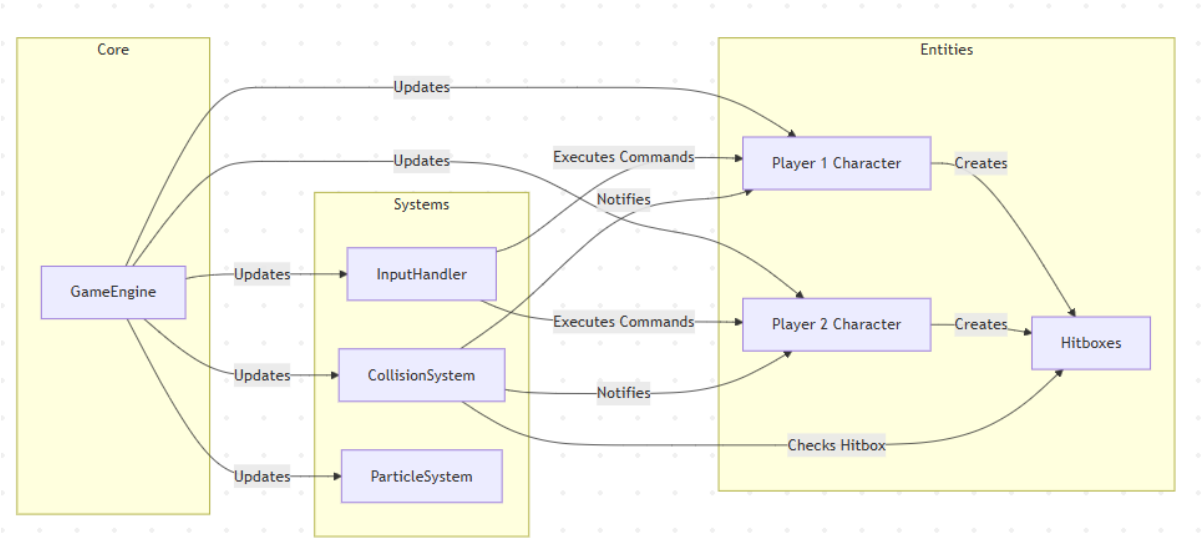
6.1 Subsystems and Modules

- **GameEngine:** Orchestrates the initialization and the main requestAnimationFrame loop.
- **InputHandler:** Maps raw browser keyboard events to specific game commands.
- **CollisionSystem:** Handles AABB (Axis-Aligned Bounding Box) logic to prevent overlaps and detect hits.
- **CharacterFactory:** Centralized module for creating player entities with specific configurations.

6.2 Responsibilities of Each Component

- **Character:** Manages individual health, position, velocity, and current state.
- **Physics Engine:** Applies gravity and friction constants to active entities.

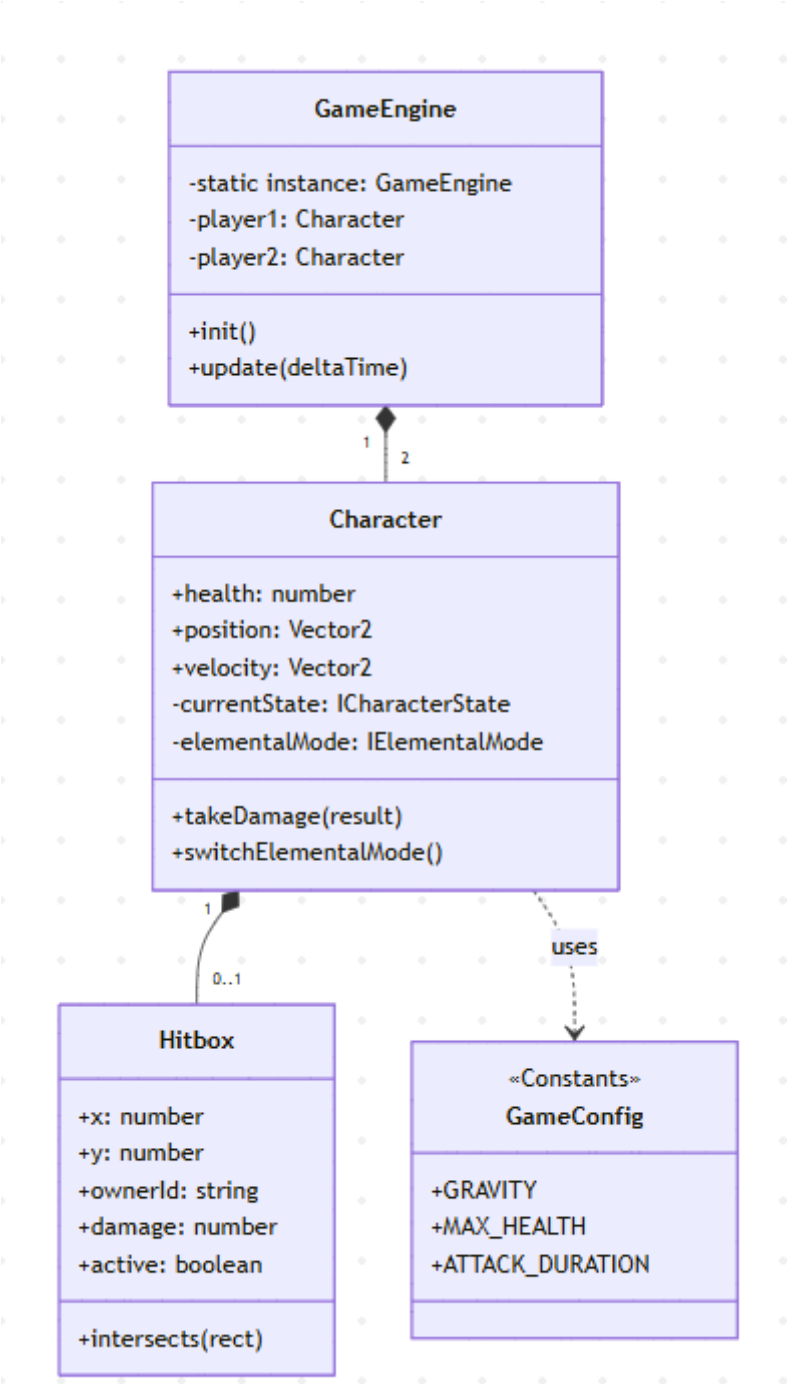
6.3 Component Diagrams



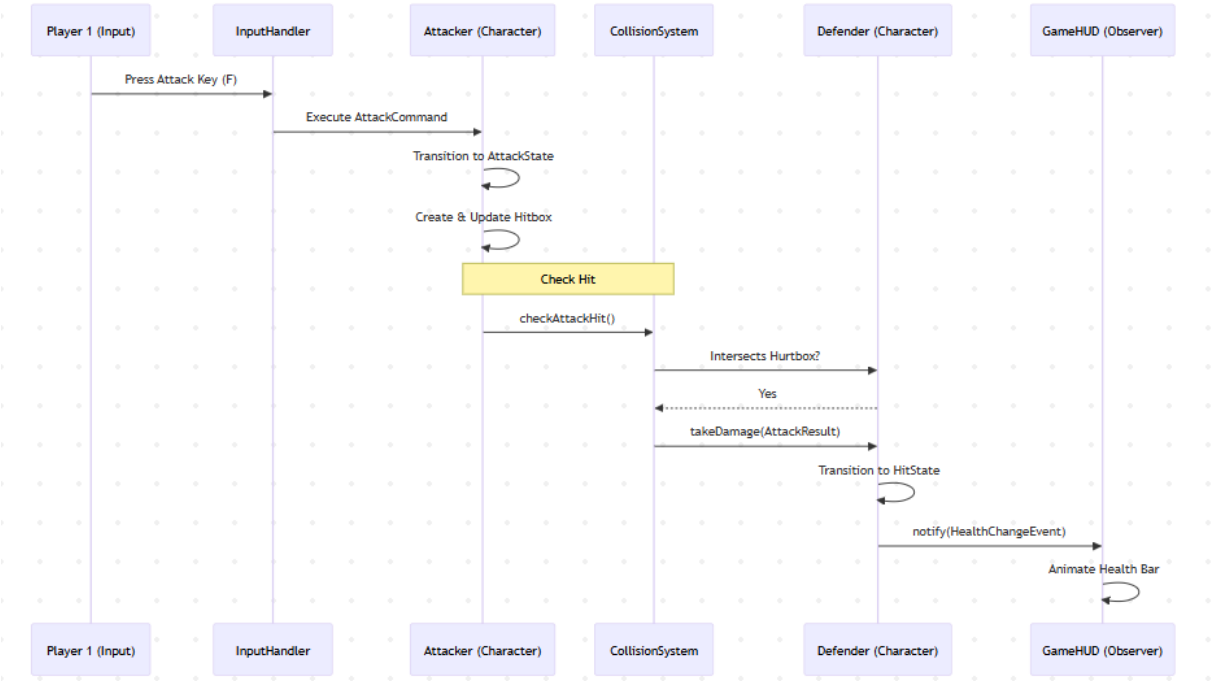
7. Data Design

7.1 Data Model

Data is managed through a central configuration and runtime class instances.



7.2 Data Flow Diagrams



8. Design Patterns

8.1 Applied Design Patterns

Singleton: The GameEngine ensures a single orchestrator for the game loop.

Strategy: IElementalMode allows dynamic switching between Fire and Water combat attributes.

State: ICharacterState manages transitions between Idle, Move, Attack, and Hit.

Command: ICommand decouples raw keyboard events from the Character logic.

Observer: Decouples the UI (GameHUD) from the core game logic, updating health bars via notifications.

Factory: CharacterFactory centralizes the creation and initialization of player characters.

Object Pool: ParticleSystem reuses particles and hitboxes to prevent performance spikes.

8.2 Context and Justification

The State Pattern is critical to ensure a character cannot perform actions while in a "stunned" (Hit) state. The Strategy Pattern allows for runtime modification of combat stats without altering the core character class.

9. Implementation Notes

Movement uses a velocity-based system with Delta-Time (dt) integration. This ensures that character speed remains consistent regardless of whether the user has a 60Hz or 144Hz monitor.

10. User Interface Design

The UI consists of a HUD (Heads-Up Display) overlaying the Canvas. It includes:

- Dynamic Health Bars (Observer-driven).
- Mode Indicators (Fire/Water icons).
- Game State overlays (Start/Game Over).

11. Performance Considerations

- **Object Pooling:** Hitboxes and particles are recycled from a pool to prevent "Garbage Collection spikes" during intense combat.
- **Culling:** Entities outside the viewport are not rendered to save GPU cycles.

12. Error Handling and Logging

- **Exception Management:** The GameEngine uses try-catch blocks during initialization to handle failures in canvas context acquisition.
- **Logging:** A custom Logger tracks state transitions and input mapping for debugging purposes.

13. Deployment and Installation Design

- **Environment:** Node.js v18+.
- **Installation:** npm install.
- **Execution:** npm run dev for local development; npm run build for production-ready static files.

14. Change Log

- **v1.0:** Initial core engine and physics implementation.
- **v1.1:** Integrated State, Command, and Strategy patterns.

15. Future Work / Open Issues

- **UC4:** Implementation of local tournament brackets.
- **Audio Engine:** Integration of Web Audio API for sound effects.

16. Task Assignment Matrix

Team Member	Contribution (Classes & Tasks)	Use Case Mapping
Mehmet Umur ÖZÜ	**Core & Input Systems**: Implemented the GameEngine singleton, InputHandler, IdleState, and the Command pattern infrastructure.	**UC1**: Start Game Battle, **UC2**: Execute Combat Actions.
Mustafa GÖZÜTOK	**Entity & Physics**: Developed the CharacterFactory, Character class, MoveState, and the CollisionSystem for AABB detection.	**UC1**: Start Game Battle, **UC2**: Execute Combat Actions.
Mehmet Yasin TOSUN	**Combat & UI**: Implemented AttackState, HitState, Hitbox management, main.ts entry point, and authored the Software Design Document (SDD).	**UC2**: Execute Combat Actions.